Laura Hietala

Oppimistehtävä 3

Tietojenkäsittelyn koulutus
TKMI23KV

1.12.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

Tehtävänä on toteuttaa t-paitojen verkkokaupan aloitusnäkymä soveltaen React-komponentteja. Pelkkä aloitusnäkymä siis riittää, koko verkkokauppaa ei tarvitse toteuttaa. Näkymän ulkoasun ja asettelun saa soveltaa vapaasti.

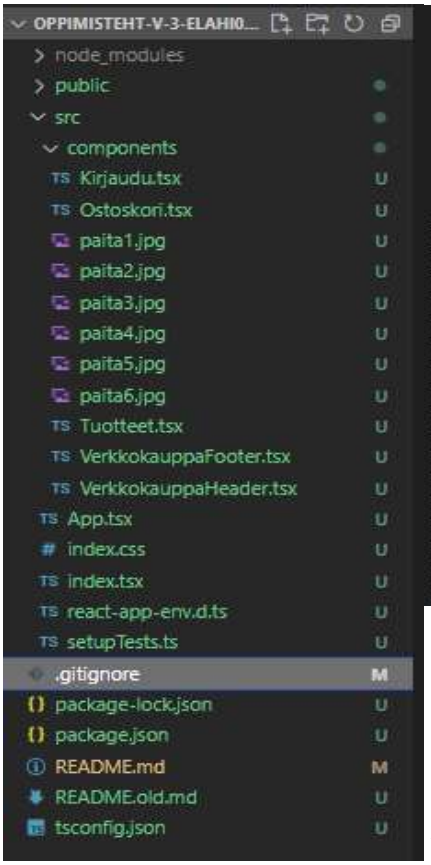
GitHub*:* *https://github.com/xamk-so1/oppimisteht-v-3-elahi001xamk*

Perusvaatimukset sovellukselle

Vaatimus: Sovellus koostuu (App-komponentin lisäksi) vähintään seuraavista komponenteista:

- VerkkokauppaHeader: verkkokaupan yleinen otsikko, mahdollinen grafiikka jne.
- VerkkokauppaFooter: verkkokaupan yhteystiedot (keksittyjä) ja copyright kuluvalle vuodelle.
- Tuotenostot: yhteensä kuusi t-paitaa sijoitettuna kahdelle riville. Jokaisesta tuotteesta on esillä nimi, kuva (etsi vapaasti verkosta) ja hinta.
- Kirjautuminen: kaksi input-kenttää (tunnus ja salasana) ja painonappi "Kirjaudu"
- Pikaostoskori: oletuksena otsikko ja teksti "Ostoskorissa ei ole tuotteita."

Ratkaisu:




```tsx
import React from 'react';
import { Box, Typography } from '@mui/material';

const VerkkokauppaHeader : React.FC = () : React.ReactElement => {

    const header = {
      boxSizing: "border-box",
      backgroundColor: "pink",
      border: "1px solid lightgrey",
      padding: "50px",
    };

    const heading = {
      color: "grey",
      textTransform: "uppercase",
      fontFamily: "Courier"
    };

    return (
      <Box sx={header} >
        <Typography variant = "h2" sx={heading}>T-paitojen verkkokauppa</Typography>
      </Box>
    )
    };

    export default VerkkokauppaHeader;
```

Vasemmalla näkymä komponenteista. Ylhäällä oikealla header muotoiluineen.

Alla footer muotoiluineen ja yhteystietoineen. Rivillä 32 on toteutettu koodi, joka näyttää tekijänoikeusmerkin sekä nykyisen vuoden.

```tsx
import React, {useState} from 'react';
import { Box, Typography, Grid, Paper, Container } from '@mui/material';


const VerkkokauppaFooter : React.FC = () : React.ReactElement => {

const footer = {
    boxSizing: "border-box",
    backgroundColor: "pink",
    border: "1px solid lightgrey",
    padding: "50px",
  };

  const heading = {
    color: "grey",
    textTransform: "uppercase",
    fontFamily: "Courier"
  };
  const getCurrentYear = () => {
    return new Date().getFullYear();
  };

return (
    <Box sx={footer}>

        <Typography variant="h4" align="center" sx={heading}>Yhteystiedot</Typography>
        <Typography variant="body1" align="center" sx={{padding:"25px", fontFamily: "Courier"}}>
            T-paitojen verkkokauppa <br />
            Paitapolku 1 <br />
            01230 Paitala</Typography>
        <Typography variant="body2" color="textSecondary" align="center">
            &copy; {getCurrentYear()} T-paitojen verkkokauppa
        </Typography>
```

Tuotteet-komponentissa myyntiartikkelit (t-paidat) on aseteltu vaaditulla tavalla hyödyntäen MUI:n Gridiä (alkaen riviltä 33):

```
import Paper from '@mui/material/Paper';
import React from 'react';
import paita1 from './paita1.jpg';
import paita2 from './paita2.jpg';
import paita3 from './paita3.jpg';
import paita4 from './paita4.jpg';
import paita5 from './paita5.jpg';
import paita6 from './paita6.jpg';
import { Grid, Typography, Button } from '@mui/material';

interface ProductListProps {
  addToCart: (productName: string, price: number) => void;
  discount: number;
}

const Tuotteet : React.FC<ProductListProps> = ({addToCart, discount})  : React.ReactElement  =>  {
  const customStyle = {
    height: 'auto',
    maxWidth: '100%',
    maxHeight: '100%',
    marginTop: '20px',
    marginBottom: '15px',
  };

    return (
      <>
      <Paper sx={{ display: "flex", flexDirection: "column", padding: "20px", boxSizing: "border-box" }}>

          <h2>Ihanat t-paitamme</h2>

        <Paper sx={{width: "100%", height: "100%", backgroundColor: "#e0e0e0"}}>

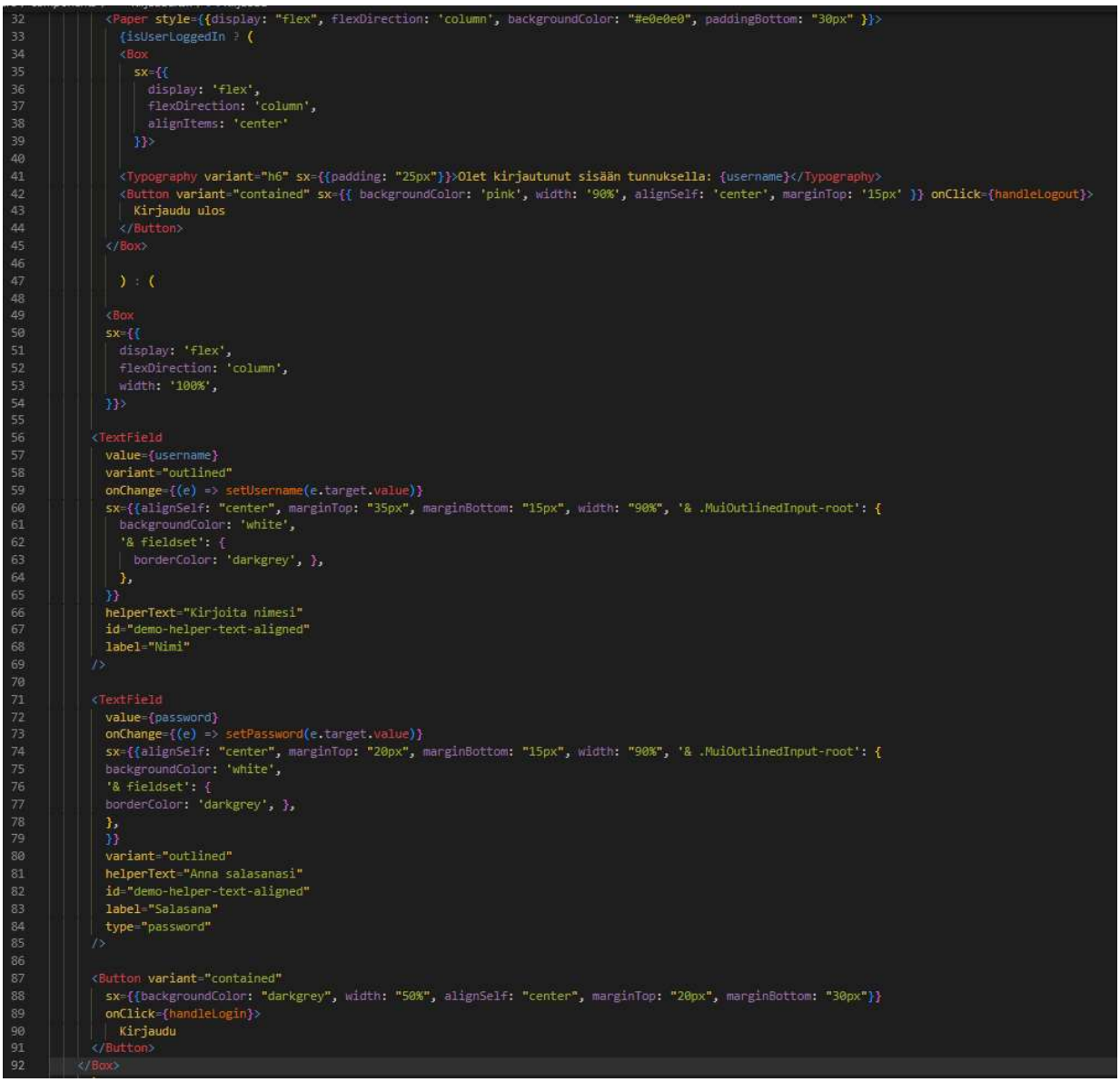
          <Grid  container spacing={2} sx={{display: "flex", flexWrap: "wrap", paddingBottom: "20px"}}>
            <Grid item xs={4}>
              <img src={paita1} alt="Paita 1" style={customStyle} />
              <Typography variant='h5' align='center'>Paita 1 / {(19.99 * (1 - discount)).toFixed(2)} €</Typography>
              <br />
              <Button variant="contained" fullWidth sx={{backgroundColor: "pink"}} onClick={() => addToCart("Paita 1", 19.99 * (1 - discount))}>
                Lisää ostoskoriin
              </Button>
            </Grid>

            <Grid item xs={4}>
              <img src={paita2} alt="Paita 2" style={customStyle} />
              <Typography variant='h5' align='center'> Paita 2 / {(19.99 * (1 - discount)).toFixed(2)} €</Typography>
              <br />
              <Button variant="contained" fullWidth sx={{backgroundColor: "pink"}} onClick={() => addToCart("Paita 2", 19.99 * (1 - discount))}>
                Lisää ostoskoriin
              </Button>
            </Grid>

            <Grid item xs={4}>
              <img src={paita3} alt="Paita 3" style={customStyle} />
              <Typography variant='h5' align='center'>Paita 3 / {(19.99 * (1 - discount)).toFixed(2)} €</Typography>
              <br />
              <Button variant="contained" fullWidth sx={{backgroundColor: "pink"}} onClick={() => addToCart("Paita 3", 19.99 * (1 - discount))}>
                Lisää ostoskoriin
              </Button>
            </Grid>

```

Seuraavassa kuvassa on osa Kirjaudu-komponentista. Kuvasta käy ilmi, että komponentti sisältää kaksi input-kenttää (tunnus ja salasana) ja painonapin "Kirjaudu".



```
<Paper style={{display: "flex", flexDirection: 'column', backgroundColor: "#e0e0e0", paddingBottom: "30px" }}>
  {isUserLoggedIn ? (
  <Box
    sx={{
      display: 'flex',
      flexDirection: 'column',
      alignItems: 'center'
    }}>

  <Typography variant="h6" sx={{padding: "25px"}}>Olet kirjautunut sisään tunnuksella: {username}</Typography>
  <Button variant="contained" sx={{ backgroundColor: 'pink', width: '90%', alignSelf: 'center', marginTop: '15px' }} onClick={handleLogout}>
    Kirjaudu ulos
  </Button>
</Box>

  ) : (

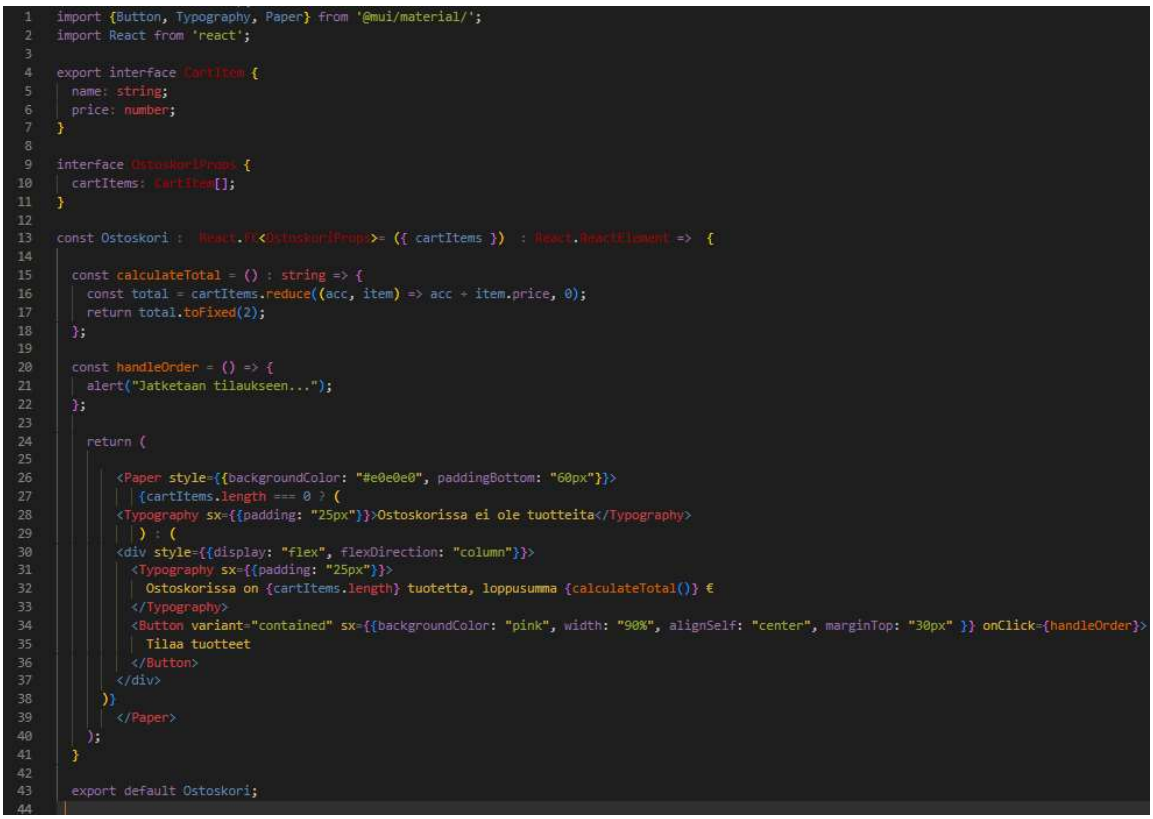
<Box
sx={{
  display: 'flex',
  flexDirection: 'column',
  width: '100%',
}}>

<TextField
  value={username}
  variant="outlined"
  onChange={(e) => setUsername(e.target.value)}
  sx={{alignSelf: "center", marginTop: "35px", marginBottom: "15px", width: "90%", '& .MuiOutlinedInput-root': {
    backgroundColor: 'white',
    '& fieldset': {
      borderColor: 'darkgrey', },
    },
  }}
  helperText="Kirjoita nimesi"
  id="demo-helper-text-aligned"
  label="Nimi"
/>

<TextField
  value={password}
  onChange={(e) => setPassword(e.target.value)}
  sx={{alignSelf: "center", marginTop: "20px", marginBottom: "15px", width: "90%", '& .MuiOutlinedInput-root': {
  backgroundColor: 'white',
  '& fieldset': {
  borderColor: 'darkgrey', },
  },
  }}
  variant="outlined"
  helperText="Anna salasanasi"
  id="demo-helper-text-aligned"
  label="Salasana"
  type="password"
/>

<Button variant="contained"
  sx={{backgroundColor: "darkgrey", width: "50%", alignSelf: "center", marginTop: "20px", marginBottom: "30px"}}
  onClick={handleLogin}>
    Kirjaudu
  </Button>
</Box>
```

Seuraavassa kuvassa on Ostoskori-komponentti. Kuvasta käy ilmi, että komponentin oletuksena on otsikko ja teksti "Ostoskorissa ei ole tuotteita."



```tsx
import {Button, Typography, Paper} from '@mui/material/';
import React from 'react';

export interface CartItem {
  name: string;
  price: number;
}

interface OstoskoriProps {
  cartItems: CartItem[];
}

const Ostoskori :  React.FC<OstoskoriProps>= ({ cartItems })  : React.ReactElement =>  {

  const calculateTotal = () : string => {
    const total = cartItems.reduce((acc, item) => acc + item.price, 0);
    return total.toFixed(2);
  };

  const handleOrder = () => {
    alert("Jatketaan tilaukseen...");
  };

    return (

        <Paper style={{backgroundColor: "#e0e0e0", paddingBottom: "60px"}}>
          {cartItems.length === 0 ? (
        <Typography sx={{padding: "25px"}}>Ostoskorissa ei ole tuotteita</Typography>
          ) : (
        <div style={{display: "flex", flexDirection: "column"}}>
          <Typography sx={{padding: "25px"}}>
            Ostoskorissa on {cartItems.length} tuotetta, loppusumma {calculateTotal()} €
          </Typography>
          <Button variant="contained" sx={{backgroundColor: "pink", width: "90%", alignSelf: "center", marginTop: "30px" }} onClick={handleOrder}>
            Tilaa tuotteet
          </Button>
        </div>
      )}
        </Paper>
    );
  }

  export default Ostoskori;
```

Vaatimus: Kunkin tuotteen yhteydessä on painonappi "Lisää ostoskoriin". Napin painaminen muuttaa pikaostoskorissa olevan tekstin muotoon "Ostoskorissa on n tuotetta, loppusumma xx e". Tuotteiden määrä ja loppusumma tekstissä määräytyy klikattujen tuotteiden mukaan.

Ratkaisu: Yllä olevassa kuvassa näkyy, miten olen toteuttanut nämä vaatimukset Ostoskorin puolella. Riveillä 15-18 määritellään funktio calculateTotal, joka laskee ostoskorin kokonaishinnan. Riviltä 24 alkaa Ostoskori-funktion palautus, jossa käytetään ternary-operaattoria palauttamaan tekstiä sen mukaan, mitä ostoskoriin on klikkailtu. Riviltä 31 riville 33 määritellään vaadittu fraasi, joka sisältää ostoskorin tuotteiden määrän (cartItems.length) sekä loppusumman, joka tulee tuosta calculateTotal-funktiosta.

Tuotteet-komponentissa (kuva s. 3) painonappi (Button) määritellään jokaisen tuotteen yhteydessä (esim. rivit 38-40, tämä toistuu samanlaisena jokaisen tuotteen kohdalla). Nappiin on lisätty onClick-tapahtumankäsittelijä, joka määritellään funktiossa addToCart App.tsx-komponentissa (kuva seuraavalla sivulla, rivit 18-21), joka on myös osa Tuotteet-funktion propseja.

addToCart-funktio saa kaksi parametria: productName ja price (rivi 18). Rivillä 19 luodaan uusi CartItem-objekti (newItem), joka koostuu productName- (tuotteen nimi) ja price-kentistä (tuotteen hinta).
Ostoskorin tilan päivittämisestä huolehtii rivillä 20 setCartItems-funktio: spredataan olemassa oleva ostoskorin sisältö uuteen taulukkoon ja lisätään uusi tuote taulukon loppuun (newItem). Napin painaminen käynnistää funktion ja lisää uuden tuotteen ostoskoriin.

```
12  const App : React.FC = () : React.ReactElement => {
13
14    const [cartItems, setCartItems] = React.useState<CartItem[]>([]);
15    const [isUserLoggedIn, setIsUserLoggedIn] = useState<boolean>(false);
16    const [discount, setDiscount] = useState<number>(0);
17
18    const addToCart = (productName: string, price: number) => {
19      const newItem: CartItem = { name: productName, price: price };
20      setCartItems([...cartItems, newItem]);
21    };
```

Vaatimus: Kun ostoskorissa on yksikin tuote, pikaostoskori-komponenttiin tulee näkyviin painonappi "Tilaa tuotteet". Napin painaminen avaa ainoastaan selaimen oman alert-ilmoituksen "Jatketaan tilaukseen...". Varsinaista tilausprosessia ei tarvitse käsitellä tämän kummemmin.

Ratkaisu: Kuva sivulla 5. Button tulee esiin, jos cartItems.length ei ole yhtä kuin 0, ja tämä toteutetaan ternary-operaattorin avulla (rivit 27-38). Riveillä 34-36 oleva Button-komponentti huolehtii Alert-ilmoituksen käynnistymisestä. Buttoniin on lisätty onClick-tapahtumankäsittelijä, joka määritellään funktiossa handleOrder (rivit 20-22). Kun käyttäjä painaa "Tilaa tuotteet" -nappia, tapahtumankäsittelijä handleOrder suoritetaan, ja se avaa ilmoituksen (alert) tekstillä "Jatketaan tilaukseen...".

Lisävaatimukset sovellukselle

Vaatimus: Kun käyttäjä syöttää kirjautumiseen tunnuksen "testi" ja salasanan "testi", vaihtuu komponentin sisällöksi teksti "Olet kirjautunut sisään tunnuksella: testi" ja sen alle painonappi "Kirjaudu ulos". Uloskirjautuminen palauttaa sisällön input-kenttiin jne. Huom. Tarkoituksena ei ole tehdä aitoa kirjautumista sessiohallintoineen, "dummy"-versio riittää. Onnistunut kirjautuminen aiheuttaa kaikkien tuotteiden hintaan 20 prosentin alennuksen. Alkuperäinen ja

alennettu hinta näkyvät tuotteiden yhteydessä. Hinnat päivittyvät myös ostoskoriin.

Ratkaisu: App-komponentissa määritellään kirjautumiseen liittyvät tilat. Rivin 15 isUserLoggedIn seuraa, onko käyttäjä kirjautunut sisään vai ei. Rivin 16 discount seuraa alennuksen prosenttiosuutta, joka päivittyy onnistuneen kirjautumisen yhteydessä.

```
12  const App : React.FC = () : React.ReactElement => {
13
14    const [cartItems, setCartItems] = React.useState<CartItem[]>([]);
15    const [isUserLoggedIn, setIsUserLoggedIn] = useState<boolean>(false);
16    const [discount, setDiscount] = useState<number>(0);
```

Alennus määritellään App-komponentissa rivillä 25 osana handleLogin-funktiota. Kirjautumiseen liittyvä funktio handleLogin (rivit 23-26) suoritetaan, kun kirjautumisnappia painetaan. Funktio myös tarkistaa, onko käyttäjätunnus ja salasana oikein ja jos on, niin funktio päivittää tiloja isUserLoggedIn ja discount.
Funktio handleLogout (rivit 28-31) suoritetaan, kun uloskirjautumisnappia painetaan. Funktio nollaa kirjautumisen ja alennuksen:

```
23    const handleLogin: KirjauduProps['onLogin'] = (username, password) => {
24      setIsUserLoggedIn(true);
25      setDiscount(0.2);
26    };
27
28    const handleLogout: KirjauduProps['onLogout'] = () => {
29      setIsUserLoggedIn(false);
30      setDiscount(0);
31    };
```

discount-tila välitetään Tuotteet-komponentille propsina (rivit 11-14):

```
11  interface ProductListProps {
12    addToCart: (productName: string, price: number) => void;
13    discount: number;
14  }
```

Jokaisen tuotteen hinnassa käytetään discount-prosenttiosuutta, kun käyttäjä on kirjautunut sisään (tässä esimerkki yhdestä tuotteesta, discount rivillä 36 ja 38):

```
36          <Typography variant='h5' align='center'>Paita 1 / {(19.99 * (1 - discount)).toFixed(2)} €</Typography>
37          <br />
38          <Button variant="contained" fullWidth sx={{backgroundColor: "pink"}} onClick={() => addToCart("Paita 1", 19.99 * (1 - discount))}>
39            Lisää ostoskoriin
40          </Button>
```

Aiemmin esitellyt handleLogin ja handleLogout käsittelevät kirjautumista ja näyttöliittymä näyttää eri sisällön riippuen sitä, onko käyttäjä kirjautunut sisään vai ei. Tästä huolehtii ternary-operaattori, joka rivillä 33 tarkistaa, onko käyttäjä kirjautunut sisään ja jos on, näytetään teksti ”Olet kirjautunut sisään tunnuksella: {username} (rivi 41) ja nappi "Kirjaudu ulos" (rivit 42-44).
Jos ei ole kirjautunut sisään, näytetään input-kentät (rivit 56-85) ja nappi "Kirjaudu" (rivit 87-91). Nappeihin on liitetty onClick-tapahtumankäsittelijät, jotka suorittavat niihin liitetyt funktiot handleLogout (rivi 42) tai handleLogin (rivi 89). Lisäksi tekstikenttiin on liitetty funktiot setUsername (rivi 59) ja setPassword (rivi 73), jotka napin painalluksesta asettavat uuden arvon username ja password-tiloille sen mukaan, mitä käyttäjä on syöttänyt input-kenttiin. Tässä käytetään onChange tapahtumankäsittelijää, joka kutsuu setUsername- tai setPassword-funktiota.

```
33          {isUserLoggedIn ? (
34          <Box
35            sx={{
36              display: 'flex',
37              flexDirection: 'column',
38              alignItems: 'center'
39            }}>
40
41          <Typography variant="h6" sx={{padding: "25px"}}>Olet kirjautunut sisään tunnuksella: {username}</Typography>
42          <Button variant="contained" sx={{ backgroundColor: 'pink', width: '90%', alignSelf: 'center', marginTop: '15px' }} onClick={handleLogout}>
43            Kirjaudu ulos
44          </Button>
45        </Box>
46
47          ) : (
48
49        <Box
50        sx={{
51          display: 'flex',
52          flexDirection: 'column',
53          width: '100%',
54        }}>
55
56      <TextField
57        value={username}
58        variant="outlined"
59        onChange={(e) => setUsername(e.target.value)}
60        sx={{alignSelf: "center", marginTop: "35px", marginBottom: "15px", width: "90%", '& .MuiOutlinedInput-root': {
61          backgroundColor: 'white',
62          '& fieldset': {
63            borderColor: 'darkgrey', },
64          },
65        }}
66        helperText="Kirjoita nimesi"
67        id="demo-helper-text-aligned"
68        label="Nimi"
69      />
70
71      <TextField
72        value={password}
73        onChange={(e) => setPassword(e.target.value)}
74        sx={{alignSelf: "center", marginTop: "20px", marginBottom: "15px", width: "90%", '& .MuiOutlinedInput-root': {
75        backgroundColor: 'white',
76        '& fieldset': {
77        borderColor: 'darkgrey', },
78        },
79        }}
80        variant="outlined"
81        helperText="Anna salasanasi"
82        id="demo-helper-text-aligned"
83        label="Salasana"
84        type="password"
85      />
86
87      <Button variant="contained"
88        sx={{backgroundColor: "darkgrey", width: "50%", alignSelf: "center", marginTop: "20px", marginBottom: "30px"}}
89        onClick={handleLogin}>
90          Kirjaudu
91      </Button>
```

Kirjaudu-komponentin tilat username ja password (rivit 13 ja 14) seuraavat käyttäjän syöttämiä tietoja. Kirjaudu-komponentin handleLogin-funktio suoritetaan kun käyttäjä painaa Kirjaudu-nappia. Rivillä 17 aloitetaan tarkistus, ovatko käyttäjänimi ja salasana oikein. Jos käyttäjä syöttää molempiin ”testi”, kirjautuminen hyväksytään.

